

Spherepop OS: A Deterministic Semantic Operating System

Formal Specification and Architectural Rationale

Flyxion

August 25, 2026

Abstract

Spherepop OS is a semantic operating system whose primary abstraction is not the process, file, or thread, but a deterministic, append-only relational event substrate. The system is designed to support collaborative, time-aware, and introspectable computation by enforcing strict causal ordering, replayability, and separation between authoritative state transitions and derived views. This document presents a formal specification of Spherepop OS, together with the rationale for each architectural choice, and establishes that the kernel’s six event types reduce to four causal primitives (Pop, Refuse, Bind, Collapse) plus a small set of structural composition rules, with one event type shown to fall genuinely outside the causal basis. Building on this reduction, the specification extends the kernel with a full complement of Linux-inspired operating-system responsibilities — process management, scheduling, a syscall boundary, a virtual object namespace, loadable modules, and capability-based permissions — each given as a proven or explicitly flagged-as-conditional composition over the causal basis, rather than as an architectural aspiration stated only in prose. Particular emphasis is placed on determinism, observational purity, the treatment of geometry and metadata as non-causal, and the precise conditions under which multiple observers’ divergent renderings of shared state remain free of contradiction. The resulting system occupies an intermediate position between an operating system kernel, a distributed database, and a semantic field theory of computation.

1 Motivation and Design Goals

Contemporary operating systems are historically organized around mutable global state, imperative control flow, and opaque side effects. While these designs have proven effective for single-user, single-machine computation, they exhibit deep structural limitations when extended to collaborative, distributed, or semantically rich environments. In particular, traditional systems struggle to provide strong guarantees of determinism, introspectability, and time-consistent reasoning.

Spherepop OS is motivated by the following design goals:

- G1. **Deterministic Replay:** Any system state must be reproducible exactly from an authoritative log.
- G2. **Total Causal Order:** All state transitions must admit a global, unambiguous ordering.
- G3. **View–Cause Separation:** Observations, visualizations, and speculative reasoning must not interfere with authoritative state.

- G4. **Semantic Primacy:** Relations, equivalence, and meaning precede processes and files as first-class concepts.
- G5. **Collaborative Introspection:** Multiple observers may join, leave, rewind, and speculate without destabilizing the system.

These goals jointly rule out large classes of conventional kernel and database architectures, motivating a log-centric, event-sourced design with strict discipline around causality and derivation.

2 Core Ontology

Definition 2.1 (Semantic Object). *A semantic object is an abstract entity identified by a stable handle. Objects have no intrinsic meaning except insofar as they participate in relations, equivalence classes, and events.*

Definition 2.2 (Relation). *A relation is a typed, directed association between two semantic objects. Relations are first-class and may carry flags or metadata.*

Definition 2.3 (Equivalence). *An equivalence relation over objects identifies multiple handles as representing a single canonical representative. Equivalence is induced by merge events and maintained via a union-find structure.*

Importantly, objects, relations, and equivalence classes exist only as consequences of event replay. There is no mutable object table independent of the log.

3 Authoritative Event Log

Axiom 3.1 (Append-Only Authority). *All authoritative state transitions in Spheredpop OS are recorded as events in a single append-only log. No other structure is permitted to introduce or modify semantic state.*

Each event is assigned a strictly increasing sequence identifier (EID) by a single authoritative arbiter. The log is therefore totally ordered.

Proposition 3.1 (Deterministic State). *Given an initial empty state and a prefix of the event log, the resulting semantic state is uniquely determined.*

Proof. All kernel transitions are pure functions of prior state and the next event. Since the log is totally ordered and immutable, replay is deterministic. $\square$

This property elevates the log from an implementation detail to the primary artifact of the system.

4 Kernel Semantics

The Spherepop kernel is a deterministic interpreter of events. It maintains derived state such as equivalence classes and relation tables, but these structures are caches, not authorities.

Definition 4.1 (Kernel). *The kernel is the minimal deterministic machine that replays events and emits derived changes (diffs). It has no external side effects and no hidden state.*

Kernel operations include:

- Object creation (POP)
- Merge (MERGE, inducing equivalence)
- Relation creation (LINK) and removal (UNLINK)
- Region collapse (COLLAPSE)
- Metadata attachment (SET_META)

Each operation is represented as a canonical event type with a fixed binary schema.

5 Formal Execution Semantics

We now make the kernel transition function explicit by presenting a small-step operational semantics for event application. The goal is not to specify implementation details, but to provide an unambiguous mathematical description of how authoritative state evolves in response to events.

5.1 Kernel State

Definition 5.1 (Kernel State). *A kernel state is a tuple*

$$\sigma = (O, U, R, M)$$

- O is a finite set of object identifiers,
- $U : O \rightarrow O$ is a union-find parent map representing equivalence,
- $R \subseteq O \times O \times \mathcal{T}$ is a multiset of typed relations,
- $M : O \rightarrow \mathcal{K} \rightarrow \mathcal{V}$ is a partial metadata map.

We write $\text{rep}_\sigma(o)$ for the canonical representative of o under U , after path compression.

5.2 Transition Relation

Kernel execution is defined by a labeled transition relation

$$\sigma \xrightarrow{e} \sigma'$$

5.3 POP (Object Creation)

$$\frac{o \notin O}{(O, U, R, M) \xrightarrow{\text{POP}(o)} (O \cup \{o\}, U[o \mapsto o], R, M)}$$

5.4 MERGE (Equivalence Induction)

$$\frac{o_1, o_2 \in O}{\sigma \xrightarrow{\text{MERGE}(o_1, o_2)} \sigma'}$$

where σ' differs from σ only in U , with

$$U'(\text{rep}(o_2)) = \text{rep}(o_1).$$

This rule is idempotent: merging already-equivalent objects yields no further change.

5.5 LINK (Relation Creation)

$$\frac{(o_1, o_2, t) \notin R}{\sigma \xrightarrow{\text{LINK}(o_1, o_2, t)} (O, U, R \cup \{(\text{rep}(o_1), \text{rep}(o_2), t)\}, M)}$$

5.6 UNLINK (Relation Removal)

$$\frac{(o_1, o_2, t) \in R}{\sigma \xrightarrow{\text{UNLINK}(o_1, o_2, t)} (O, U, R \setminus \{(\text{rep}(o_1), \text{rep}(o_2), t)\}, M)}$$

5.7 COLLAPSE (Bulk Equivalence)

Let $S \subseteq O$ be a region to be collapsed, with chosen representative $o_r \in S$.

$$\frac{S \subseteq O}{\sigma \xrightarrow{\text{COLLAPSE}(S, o_r)} \sigma'}$$

where for all $o \in S$, $U'(\text{rep}(o)) = \text{rep}(o_r)$. Objects not equal to o_r remain in O but are no longer representatives.

5.8 SET_META (Metadata Attachment)

$$\frac{o \in O}{\sigma \xrightarrow{\text{SET_META}(o, k, v)} (O, U, R, M[o \mapsto (k \mapsto v)])}$$

Metadata updates do not affect O , U , or R .

5.9 Semantic Properties

Proposition 5.1 (Determinism). *For any kernel state σ and event e , there exists at most one state σ' such that $\sigma \xrightarrow{e} \sigma'$.*

Proposition 5.2 (Merge Confluence). *The final equivalence relation induced by a set of MERGE events is independent of their order.*

Proof. Union–find union is associative and commutative over equivalence classes. The induced partition is given by the reflexive–transitive closure of the merge relation and therefore does not depend on merge order. $\square$

Proposition 5.3 (View Consistency). *Derived views that collapse non-representative objects preserve semantic equivalence.*

5.10 MERGE-Induced Relation Normalization

We now make explicit the interaction between equivalence induction and relations.

Axiom 5.1 (Representative Normalization). *All relations are stored and interpreted in representative-normalized form. That is, for any relation $(o_1, o_2, t) \in R$, it is invariant that $o_1 = \text{rep}(o_1)$ and $o_2 = \text{rep}(o_2)$.*

Proposition 5.4 (Relation Rewriting under MERGE). *When an event $\text{MERGE}(o_a, o_b)$ is applied, all relations incident to $\text{rep}(o_b)$ are rewritten to reference $\text{rep}(o_a)$.*

Proof. After MERGE, $\text{rep}(o_b) = \text{rep}(o_a)$. Any relation previously referencing $\text{rep}(o_b)$ is therefore normalized to reference $\text{rep}(o_a)$. No semantic information is lost, since o_a and o_b are equivalent by definition. $\square$

6 Replay Equivalence

We now formalize the equivalence between full replay and incremental observation.

Theorem 6.1 (Replay Equivalence). *Let $\ell = e_1, e_2, \dots, e_n$ be an event sequence. Let σ_{replay} be the state obtained by replaying ℓ from the initial state σ_0 . Let σ_{diff} be the state obtained by applying the sequence of diffs derived from $e_1, \dots, e_n$ to σ_0 . Then σ_{replay} and σ_{diff} are semantically equivalent.*

Proof. Each diff is a deterministic function of a single event application. Since kernel transitions are deterministic and diffs are derived without side effects, incremental application of diffs reconstructs the same representative-normalized state as direct replay of the log. Equivalence follows by induction on the length of ℓ . $\square$

Proposition 6.1 (Merge Confluence). *The final equivalence relation induced by a set of MERGE events is independent of their order.*

Proposition 6.2 (View Consistency). *Derived views that collapse non-representative objects preserve semantic equivalence.*

7 Meta-Theorem: Functoriality of Derived Views

The core philosophical discipline of Spherepop OS is the separation of *cause* (events) from *views* (snapshots, diffs, geometry). The following meta-theorem states this separation as a functoriality principle: any admissible view is a structure-preserving map out of the event-sourced semantics.

7.1 Event Prefix Category

Definition 7.1 (Prefix Category). *Fix an event log $\ell = (e_1, e_2, \dots)$. Define the prefix category $\mathbf{Pref}(\ell)$ as follows.*

- *Objects are natural numbers $n \in \mathbb{N}$, interpreted as prefixes $\ell_{\leq n} := (e_1, \dots, e_n)$.*
- *There is a unique morphism $m_{n \rightarrow n+k} : n \rightarrow n+k$ for each $k \geq 0$, representing prefix extension by k events.*
- *Composition is given by addition: $m_{n \rightarrow n+k} \circ m_{n+k \rightarrow n+k+j} = m_{n \rightarrow n+k+j}$.*

7.2 State Semantics as a Functor

Definition 7.2 (State Semantics). *Let $\mathbf{State}$ be the category whose objects are kernel states σ and whose morphisms are deterministic state updates induced by event application. Define the state semantics functor*

$$S_\ell : \mathbf{Pref}(\ell) \rightarrow \mathbf{State}$$

by $S_\ell(n) := \sigma_n$, the state obtained by replaying $\ell_{\leq n}$ from σ_0 , and by mapping each generator $m_{n \rightarrow n+1}$ to the single-step transition

$$\sigma_n \xrightarrow{e_{n+1}} \sigma_{n+1}.$$

Proposition 7.1 (Functoriality of Replay). *The assignment S_ℓ is a functor.*

Proof. Identity morphisms correspond to applying zero events and therefore map to identity state updates. Composition in $\mathbf{Pref}(\ell)$ corresponds to sequential event application, which equals composition of deterministic transitions in $\mathbf{State}$ by definition. $\square$

7.3 Derived Views

Definition 7.3 (Admissible View). *An admissible view is any functor*

$$V : \mathbf{State} \rightarrow \mathbf{View}$$

into some category of observer representations $\mathbf{View}$ (e.g. JSON graphs, NDJSON diffs, geometric layouts), such that V is non-interfering: it does not feed back into S_ℓ .

Theorem 7.1 (View Functoriality Meta-Theorem). *For any log ℓ and any admissible view functor $V : \mathbf{State} \rightarrow \mathbf{View}$, the composite*

$$V \circ S_\ell : \mathbf{Pref}(\ell) \rightarrow \mathbf{View}$$

exists and is a functor. In particular:

- (i) **Causal Respect:** view updates compose according to event order;
- (ii) **Snapshot Coherence:** $V(\sigma_n)$ is uniquely determined by the prefix $\ell_{\leq n}$;
- (iii) **Transport Independence:** any two admissible transports realizing the same V are observationally equivalent;
- (iv) **Gauge Freedom:** if V quotients by a metadata gauge (e.g. layout hints), semantic content is preserved.

Proof. Since S_ℓ is a functor and V is a functor, their composite is a functor. Items (i)–(iv) are immediate restatements: functoriality enforces respect for composition (event order), object mapping enforces prefix-determined snapshots, transport choices refine the same arrow in **View**, and gauge quotienting corresponds to functoring into a category that identifies gauge-equivalent metadata. $\square$

Corollary 7.1 (Diffs and Snapshots as Two Views). *Let V_{snap} be the view that serializes full state (snapshot) and V_{diff} be the view that serializes incremental changes. Both are admissible views, hence both factor functorially through S_ℓ . The Replay Equivalence theorem is the statement that these two views agree on semantic content up to the observer’s equivalence relation.*

Corollary 7.2 (Late-Joiner Correctness). *A late-joining observer that first receives $V_{\text{snap}}(\sigma_k)$ and then receives V_{diff} for subsequent events reconstructs the same view as an observer that applied V_{diff} from the beginning.*

8 Diffs and Incremental Observation

While the log is authoritative, clients require efficient incremental observation.

Definition 8.1 (Diff). *A diff is a derived, non-authoritative description of how kernel state changes as a result of applying a single event.*

Diffs may include object additions, removals, relation changes, or metadata updates. They are broadcast to observers but are never logged themselves.

Axiom 8.1 (View Non-Interference). *Diffs do not influence kernel state and may be dropped, reordered, or ignored by observers without affecting correctness.*

This separation permits high-frequency visualization without compromising determinism.

9 Snapshots as Derived Views

Definition 9.1 (Snapshot). *A snapshot is a complete serialization of kernel state derived by replaying a prefix of the event log.*

Snapshots are used for bootstrapping late-joining clients and for historical inspection.

Proposition 9.1 (Snapshot Purity). *Snapshots introduce no new information beyond what is contained in the log prefix they represent.*

Snapshots are never logged and never assigned sequence identifiers. They are observational artifacts only.

10 Seekable Time and Historical Inspection

Spherepop OS supports time navigation by allowing clients to request snapshots at arbitrary past EIDs.

Definition 10.1 (Seek-to-EID Snapshot). *A seekable snapshot is a snapshot derived by replaying the log up to a specified EID.*

This operation is implemented using a temporary kernel instance, ensuring isolation from live state.

Corollary 10.1. *Historical inspection cannot affect the present state of the system.*

11 Speculative Branches

Speculation is treated as a local, non-authoritative overlay.

Definition 11.1 (Speculative Branch). *A speculative branch consists of a base EID and a client-local overlay log of hypothetical events.*

Branches are replayed by applying the authoritative log up to the base EID, followed by the overlay events. Branches may be discarded or rebased freely.

Axiom 11.1 (Speculative Isolation). *Speculative events have no effect on authoritative state unless explicitly re-submitted as proposals.*

This design permits safe exploration without timeline pollution.

12 Layout and Geometry as Metadata

Spherepop OS distinguishes sharply between semantic structure and geometric presentation.

Definition 12.1 (Layout Hint). *A layout hint is non-causal metadata attached to an object, expressing advisory geometric information such as position, scale, or clustering intent.*

Layout hints may be modified freely by clients and are never interpreted by the kernel as semantic operations.

Proposition 12.1. *Geometry in Spherepop OS is a gauge choice, not a semantic constraint.*

This permits rich visualization while preserving semantic invariants.

13 Arbiter and Collaboration Model

The arbiter is responsible for assigning sequence identifiers and appending events to the log.

Definition 13.1 (Arbiter). *The arbiter is the sole authority permitted to commit events to the log.*

Clients submit proposals, which are either accepted and sequenced or rejected. Accepted proposals are broadcast as diffs to all observers, including the originator.

Axiom 13.1 (Single Sequencer). *There exists exactly one arbiter per authoritative log.*

This ensures total causal order.

14 Reduction of OS Events to the Causal Calculus

The preceding sections specified six kernel event types — POP, MERGE, LINK, UNLINK, COLLAPSE, SET_META — as primitive to the operational semantics of Section 5. This section establishes that only two of them, POP and COLLAPSE, are primitive in the stronger sense of corresponding directly to Spherepop’s underlying causal calculus, whose four operators are Pop, Refuse, Bind, and Collapse. The remaining four kernel events are typed compositions over that calculus, and one of them, SET_META, is not a composition over it at all. Stating this precisely narrows the completeness claim this specification has been informally trading on: the four-operator calculus is complete for causal history transformation, not for every event the kernel exposes.

14.1 The Term Grammar, Extended

The causal calculus’s term grammar types judgments of the form $\Gamma \vdash t : \text{State}$ under rules for state-introduction, Bind, Pop, Refuse, Collapse, and sequential composition. That grammar is single-history throughout: no rule combines two independently constructed terms t_1, t_2 into one. LINK and MERGE both require exactly this, and the calculus’s own framing — histories compose via the tensor product of a strict monoidal category — already asserts the operation exists without supplying its typing rule. We supply it here.

Axiom 14.1 (Tensor Composition). *For $\Gamma \vdash t_1 : \text{State}$ and $\Gamma \vdash t_2 : \text{State}$,*

$$\frac{\Gamma \vdash t_1 : \text{State} \quad \Gamma \vdash t_2 : \text{State}}{\Gamma \vdash t_1 \otimes t_2 : \text{State}} \quad (\text{tensor})$$

$t_1 \otimes t_2$ denotes the joint state whose option space and history are the monoidal composition of t_1 ’s and t_2 ’s, in the sense already named, but not formalized, in the companion essay’s discussion of concurrency as composition of histories.

Proposition 14.1 (Tensor Is Structural, Not Causal). *$\otimes$ is not a fifth causal operator alongside Pop, Refuse, Bind, and Collapse. It performs no selection, rejection, narrowing, or projection on any option space; it constructs the multi-history term on which those four operators subsequently act. Its addition to the grammar does not enlarge the causal algebra, in the same sense that the pre-existing sequence rule does not.*

Proof. Immediate from Axiom 14.1's premises and conclusion sharing the same type **State**: $\otimes$ introduces no new judgment form, no new observable type, and no rule targeting **Opt** or H directly. It is a structural rule in the same sense sequence is, differing only in combining two terms rather than ordering one term after another. $\square$

14.2 POP and COLLAPSE as Already Primitive

Proposition 14.2 (POP Reduces Trivially). *Kernel event $\text{POP}(o)$, object creation, is the causal term $\text{Pop}_{\text{exists}}(\text{Bind}_{\text{true}}(\{\text{exists}_o, \text{absent}_o\}))$: a trivial Bind over the unconditionally admissible two-element option space $\{\text{exists}, \text{absent}\}$, followed by a Pop committing existence. No structural composition is required, since object creation concerns a single new identity rather than a relation between existing ones.*

Proposition 14.3 (COLLAPSE Reduces to Collapse Plus an Identity-Designation Step). *Kernel event $\text{COLLAPSE}(S, o_r)$ is $\text{Collapse}(\bigotimes_{o \in S} H_o)$ — an instance of the causal Collapse operator applied to a tensor-composed region history — together with an external identity-designation step selecting o_r as the representative for every $o \in S$, recorded in the kernel-state update $U'(\text{rep}(o)) = \text{rep}(o_r)$ (Section 5.7).*

Proof. The projection $\text{Collapse}(\bigotimes_{o \in S} H_o)$ accounts for the observable value produced by bulk collapse, exactly as the causal Collapse operator is defined. It does not account for U' 's reassignment of every $o \in S$'s representative to o_r : by Proposition 14.1, $\otimes$ is silent on identity, and nothing in the causal Collapse operator's own definition (Definition 19.1, Axiom 19.1) specifies which of several composed objects' identities should be designated canonical afterward. Choosing o_r is therefore an external policy decision, structurally identical to the one Proposition 14.6 already found necessary for MERGE, here generalized from a pair to an arbitrary finite S . $\square$

Corollary 14.1 (COLLAPSE Is Hybrid, Not Directly Primitive). *Kernel event $\text{COLLAPSE}(S, o_r)$ is not directly primitive in the sense Proposition 14.2 shows POP to be. It is derived, combining the causal Collapse operator, the structural tensor rule, and an identity-designation policy external to both — the same three-part hybrid structure Proposition 14.6 established for MERGE, not a clean single-primitive reduction.*

Proof. Immediate from Proposition 14.3: the kernel event's full behavior requires the identity-designation step in addition to the causal Collapse operator and tensor composition, so no single causal primitive alone accounts for it, unlike POP (Proposition 14.2). $\square$

This corrects a classification error in an earlier draft of this reduction table, which treated kernel COLLAPSE as directly primitive alongside POP. It is not: only POP is primitive among the six kernel event types without further qualification.

14.3 LINK

Definition 14.1 (LINK Reduction).

$$\text{LINK}(a, b, r) := \text{Collapse}\left(\text{Refuse}_{r'}\left(\text{Pop}_{r(a,b)}\left(\text{Bind}_{C_r}(H_a \otimes H_b)\right)\right)\right)$$

where C_r is the admissibility condition for relation type r , and r' ranges over any relation candidates excluded from the current option space by the choice of $r(a, b)$ (empty when r is not exclusive with any sibling relation type).

Proposition 14.4 (LINK Well-Typedness). *LINK(a, b, r) is well-typed under Axiom 14.1 and the bind, pop, refuse, and collapse rules of the causal calculus, given $\Gamma \vdash H_a : \text{State}$ and $\Gamma \vdash H_b : \text{State}$.*

Proof. By Axiom 14.1, $H_a \otimes H_b : \text{State}$. Bind applies C_r to the resulting option space, typed by the bind rule. Pop commits $r(a, b)$ from the filtered space, typed by the pop rule provided $r(a, b)$ is admissible under C_r . Refuse discharges any excluded sibling r' against the original unfiltered option space, per the causal calculus's stated division between Bind's filtering and Refuse's recording. Collapse projects the result to **Obs**, typed by the collapse rule. Each step is licensed by an existing or newly supplied grammar rule; no untyped step occurs. $\square$

14.4 UNLINK

Definition 14.2 (UNLINK Reduction).

$$\text{UNLINK}(a, b, r) := \text{Collapse}\left(\text{Pop}_{\text{revoke}_r}\left(\text{Bind}_{f_{\text{valid}}}\left(H_{a,b} \oplus \{\text{revoke}_r, \text{retain}_r\}\right)\right)\right)$$

where $H_{a,b}$ is the joint history already containing the prior $\text{Pop}_{r(a,b)}$ from a previous LINK, and $\{\text{revoke}_r, \text{retain}_r\}$ is a freshly opened option space concerning the relation's continued standing.

Proposition 14.5 (UNLINK Does Not Erase). *UNLINK(a, b, r) never removes, rewrites, or targets the earlier $\text{Pop}_{r(a,b)}$ event. $H_{a,b}$ after UNLINK strictly contains every event it contained before, plus $\text{Pop}_{\text{revoke}_r}$.*

Proof. Refuse's typing rule requires its target to lie in the *current* option space Opt_t ; an event already committed via Pop has left Opt_t and entered H_t , and is therefore not a legal target for Refuse under the grammar as given. Definition 14.2 accordingly does not attempt to Refuse the original Pop; it opens a disjoint, freshly introduced option space and commits within that. No rule in the grammar permits deletion from or rewriting of H_t ; append is the only operation the grammar exposes on history. $\square$

Which of two conflicting Pop events concerning the same relation slot is reported in the operational (i.e. Collapsed) state is a modeling choice, not a fact forced by the four primitives — exactly as the companion essay's own worked example leaves the collapse rule uninstantiated at the level of the core grammar. This specification adopts *most-recent-event precedence* as its collapse rule for relation slots: a Collapse over $H_{a,b}$ reports the relation as active if the most recent event tagged $r(a, b)$ or revoke_r in $H_{a,b}$ is a Pop of the former, and inactive if it is a Pop of the latter. This rule is stated here as an explicit application-layer choice, not derived from Pop/Refuse/Bind/Collapse themselves.

14.5 MERGE

Definition 14.3 (MERGE Reduction).

$$\text{MERGE}(o_1, o_2) := \text{Collapse}\left(\text{Refuse}_{\text{distinct}}\left(\text{Bind}_{f_{\text{compat}}}(H_{o_1} \otimes H_{o_2})\right)\right)$$

where f_{compat} filters the composed option space to continuations consistent with treating o_1 and o_2 as identical, and $\text{Refuse}_{\text{distinct}}$ records the elimination of the alternative in which they remain separate.

Proposition 14.6 (MERGE Requires Identity Absorption Beyond Tensor). *Definition 14.3 alone is insufficient to specify MERGE’s effect on kernel state, because $\otimes$ (Axiom 14.1) composes two histories without asserting anything about the immutable identities $I(o_1)$ and $I(o_2)$ underlying them. MERGE additionally requires that one representative’s identity be designated to resolve future references to both, matching the union–find update $U'(\text{rep}(o_2)) = \text{rep}(o_1)$ already given for the MERGE rule in Section 5.4.*

Proof. By Axiom 14.1, $H_{o_1} \otimes H_{o_2}$ is well-typed and yields a single composed **State**, but tensor’s definition places no constraint on I ; it is silent on identity by construction, since it is a structural rule (Proposition 14.1) rather than a causal one. A term satisfying Definition 14.3 can therefore be well-typed while leaving unresolved which of $I(o_1)$, $I(o_2)$ a future LINK or query should resolve through. Some additional assignment, external to the four primitives and to $\otimes$, is required to designate the surviving representative — precisely the role Section 5.4’s union–find update already plays operationally, here made explicit as a necessary supplement to, not a consequence of, the causal reduction. $\square$

14.6 SET_META

Proposition 14.7 (SET_META Is Outside the Causal Algebra). *SET_META(o, k, v) admits no well-typed reduction to a composition of Pop, Refuse, Bind, Collapse, and $\otimes$.*

Proof. Every rule in the causal grammar — bind, pop, refuse, collapse, sequence, and tensor (Axiom 14.1) — is defined over **State** terms whose meaning is given in terms of an option space and a history. By Section 5’s own kernel semantics, SET_META(o, k, v) updates M alone and leaves O , U , and R unchanged, meaning it modifies no option space and appends nothing causally observable to any H_o . Forcing a reduction would require introducing a fictitious option space with a single admissible candidate purely to give the event a place in the grammar, which is exactly the “hidden state or new semantics” the completeness claim is required not to smuggle in. No such reduction is therefore given, and none should be. $\square$

This is consistent with, rather than a defect of, the kernel state definition $\sigma = (O, U, R, M)$ given in Section 5.1: M was already kept structurally separate from (O, U, R) , and Section 13’s treatment of Layout Hints as “a gauge choice, not a semantic constraint” already anticipates exactly this exclusion for a different class of metadata.

14.7 The Boundary, Stated

Theorem 14.1 (Scope of the Four-Primitive Completeness Claim). *Spherepop OS has four primitive causal operators: Pop, Refuse, Bind, and Collapse. Of the six kernel event types, only POP is directly primitive without qualification. LINK and UNLINK are typed compositions over histories constructed using structural rules — sequence and tensor — that are not themselves causal operators (Proposition 14.1). MERGE and kernel COLLAPSE are hybrid: each requires that same structural composition plus an identity-designation step external to the four primitives and to tensor alike (Propositions 14.6, 14.3) — kernel COLLAPSE, despite sharing a name with the causal Collapse primitive, is not a direct instance of it, but the n -ary generalization of MERGE. SET_META is not derived from the four primitives at all, because it operates on the non-causal annotation channel M , outside the option-space algebra (Proposition 14.7). The four-primitive completeness claim therefore applies to causal history transformation, not to every operation exposed by the operating-system interface, and applies without qualification to only one of the six kernel event types.*

Proof. Immediate from Propositions 14.2, 14.3 and its Corollary 14.1, 14.4, 14.5, 14.6, and 14.7, which jointly classify all six kernel event types: one primitive without further qualification (POP); two derived via structural composition alone (LINK, UNLINK); two hybrid, requiring structural composition plus an identity-designation step external to the causal algebra proper (MERGE, COLLAPSE); and one excluded outright (SET_META). $\square$

15 Branch as a Structural Rule

Section 14 supplied tensor (Axiom 14.1) as the structural rule composing independently constructed histories into one term. fork and speculative execution both need the reverse relationship: one existing history giving rise to two, sharing an immutable prefix and diverging afterward. Nothing in the grammar as extended so far supplies this. This section adds it as its own structural rule, Branch, deliberately not reusing tensor and deliberately not typed as an unconstrained Cartesian pair.

15.1 The Branch Judgment

Definition 15.1 (Branch). *Branch introduces a distinguished judgment form rather than an ordinary product type:*

$$\frac{\Gamma \vdash t : \text{State}}{\Gamma \vdash t \Longrightarrow_e (t_p, t_c) : \text{Branch}} \quad (\text{branch})$$

with elimination rules recovering each projection as an ordinary state term:

$$\frac{\Gamma \vdash (t_p, t_c) : \text{Branch}}{\Gamma \vdash \pi_p(t_p, t_c) : \text{State}} \quad \frac{\Gamma \vdash (t_p, t_c) : \text{Branch}}{\Gamma \vdash \pi_c(t_p, t_c) : \text{State}}$$

The subscript e names the branch event distinguishing this application of the rule from any other branching of the same t .

The judgment form exists precisely so that Branch can carry an invariant $\text{State} \times \text{State}$ cannot: that t_p and t_c are not an arbitrary pair of states but two continuations of one prior history. The

typing rules alone do not yet enforce this — they only give the pair a place to be typed correctly. The enforcement is a separate axiom.

Axiom 15.1 (Branch Prefix Coherence). *If $\Gamma \vdash t \Longrightarrow_e (t_p, t_c) : \text{Branch}$, then*

$$H_{t_p} \upharpoonright_{|H_t|} = H_{t_c} \upharpoonright_{|H_t|} = H_t.$$

Both projections' histories, truncated to the length H_t had at the moment of branching, equal H_t exactly. Events appended after the branch point are unconstrained by this axiom and may differ freely between t_p and t_c .

Without Axiom 15.1, Definition 15.1 would license exactly the unconstrained pairing this section exists to avoid, merely relocated behind a new type name. With it, “Branch” names a real invariant rather than asserting one.

15.2 Identity Is Left Open by Branch

Proposition 15.1 (Branch Does Not Resolve Identity). *Definition 15.1 and Axiom 15.1 impose no constraint on $I(t_p)$ or $I(t_c)$ relative to $I(t)$.*

Proof. Neither the branch rule nor the coherence axiom mentions I ; both are stated entirely in terms of **State** typing and H . This is the same gap Proposition 14.6 identified for tensor: a structural rule concerning histories is silent on identity by construction, and silence is not a default value. $\square$

This silence is deliberate rather than an oversight to be patched uniformly, because Branch serves two consumers with opposite identity requirements. `fork` (Section 16) needs π_p to inherit $I(t)$ unchanged — the parent continues under its existing identity — while π_c receives a freshly minted identity distinct from every identity currently in scope, since the child is a new process. Speculative execution needs the opposite policy: both π_p and π_c retain $I(t)$, because a speculative branch is an alternative continuation of the *same* object under consideration, not the creation of a second one. A single Branch Identity Axiom fixed at this level of generality would be wrong for one of these two consumers no matter which policy it chose. Identity policy is therefore supplied at each use site, not by Branch itself; Section 16 states `fork`'s policy explicitly when `fork` is defined.

15.3 Branch and Tensor Are Not Inverse

Proposition 15.2 (Non-Invertibility). *For $\Gamma \vdash t \Longrightarrow_e (t_p, t_c) : \text{Branch}$, $t_p \otimes t_c \neq t$ in general.*

Proof. By Axiom 15.1, H_{t_p} and H_{t_c} agree with H_t only up to $|H_t|$; each may accumulate independent events beyond that point once $\pi_p(t_p, t_c)$ and $\pi_c(t_p, t_c)$ are used separately. Tensor (Axiom 14.1) composes t_p and t_c into a joint state whose history is their monoidal composition, containing both post-branch extensions rather than discarding either. Whenever either projection has accumulated at least one event beyond the branch point, $H_{t_p \otimes t_c}$ strictly contains H_t , so $t_p \otimes t_c \neq t$. Equality holds only in the degenerate case where neither projection has yet been extended, which is not the general case this proposition concerns. $\square$

Branch and tensor have opposite arities — Branch : $1 \rightarrow 2$ against $\otimes : 2 \rightarrow 1$ — and it is tempting on that basis alone to call them inverse. Proposition 15.2 shows this is the wrong reading. Composing two branches back together does not undo the branch; it produces a new state recording both continuations, their shared ancestry, and their divergence, which is a richer object than the pre-branch t , not a return to it. wait (Section 16) depends on exactly this: it recomposes parent and child via $\otimes$ specifically *because* that recomposition preserves what each side did independently, rather than erasing it to reconstruct a single prior state that no longer corresponds to what either process has since done.

15.4 Relation to Speculative Branches

The Speculative Branch mechanism specified in Section 11 — a base EID plus a client-local overlay log, retained until explicitly resubmitted or discarded — is Definition 15.1 under a different name, arrived at operationally before this section gave it a typing discipline. The base EID is the branch point at which Axiom 15.1 anchors agreement; the authoritative continuation and the speculative overlay are π_p and π_c ; and Axiom 11.1 (Speculative Isolation) — no effect on authoritative state unless resubmitted — is the statement that π_c remains an unresolved projection of an unresolved Branch term until some later event promotes it. Nothing in this section changes that mechanism’s behavior; it gives the mechanism a place in the calculus rather than leaving it as a kernel-level convenience with no corresponding causal term.

16 Process Management as Derived Kernel Functions

Process management is the first derived-function layer to draw on both structural rules established so far: tensor (Section 14) for wait, and Branch (Section 15) for fork. Per Theorem 14.1’s stated boundary, nothing in this section enlarges the causal basis; every operation defined here is a typed composition over Pop, Refuse, Bind, Collapse, sequence, and tensor/Branch.

16.1 Process

Definition 16.1 (Process). *A process is a tuple $\pi = (I_\pi, H_\pi, \text{Opt}_\pi, \text{Res}_\pi)$: an immutable identity I_π , a causal history H_π , a current admissible option space Opt_π , and a set of owned resource objects $\text{Res}_\pi \subseteq O$. The process’s current observable state is $\text{Collapse}(H_\pi)$.*

This is Definition 21.5’s object of study given a concrete inhabitant: a process is exactly the kind of continuing State term the causal calculus was built to describe, with Res_π naming which objects in O the process’s subsequent Binds and Pops are entitled to act on.

16.2 fork

Axiom 16.1 (Fork Identity). *For $H_\pi \Longrightarrow_e (t_p, t_c) : \text{Branch}$ invoked by fork, $I(\pi_p(t_p, t_c)) = I_\pi$ unchanged, and $I(\pi_c(t_p, t_c)) = I_{\text{new}}$, a freshly assigned identity distinct from every identity currently in scope.*

This is the use-site policy Proposition 15.1 left open: fork is the consumer of Branch for which the parent continues under its existing identity and the child is genuinely new.

Definition 16.2 (fork). $\text{fork}(\pi)$ invokes $H_\pi \Longrightarrow_e (t_p, t_c) : \text{Branch}$, and produces two process records:

$$\pi' = (I_\pi, \pi_p(t_p, t_c), \text{Opt}_\pi, \text{Res}_\pi), \quad \pi_{\text{child}} = (I_{\text{new}}, \pi_c(t_p, t_c), \text{Opt}_\pi, \text{Res}_\pi)$$

per Axiom 16.1, both option spaces and resource sets initialized identically at the branch point.

Proposition 16.1 (Resource Visibility at the Branch Point). *Every $o \in \text{Res}_\pi$ at the moment of fork is visible, with identical history, to both π' and π_{child} immediately after forking; subsequent Binds or Pops asserting exclusive ownership over any such o may diverge independently between the two.*

Proof. Immediate from Axiom 15.1: H_{t_p} and H_{t_c} agree with H_π up to $|H_\pi|$, and every $o \in \text{Res}_\pi$ has its history up to the branch point recorded within H_π by Definition 16.1. Divergence after the branch point follows from Proposition 15.2, which places no constraint on events appended past $|H_\pi|$. $\square$

Resource *reconciliation* after divergent post-fork ownership claims — what happens if both π' and π_{child} independently Bind exclusive access to the same $o \in \text{Res}_\pi$ — is not resolved by this proposition and is left to the scheduler’s arbitration (Section 17), which is the general mechanism for adjudicating competing Binds regardless of whether the competitors arose from a common fork.

16.3 exec

Definition 16.3 (exec). $\text{exec}(\pi, \text{img}) := (I_\pi, \text{Pop}_{\text{image}=\text{img}}(H_\pi), \text{Opt}_{\text{img}}, \text{Res}_\pi)$, replacing π ’s option space with the one appropriate to the new image while appending, not replacing, its history.

Proposition 16.2 (exec Does Not Erase Prior Image History). H_π after exec strictly contains H_π before it.

Proof. Immediate from the pop rule: $\text{Pop}_a(t)$ appends a to H_t and does not modify or remove any existing entry, by the same argument as Proposition 14.5. $\square$

I_π is unchanged by exec, matching the POSIX convention that exec preserves process identity while replacing the executing image — here made structurally forced rather than a convention the implementation happens to follow, since nothing in the grammar permits I to be reassigned mid-history.

16.4 exit

Definition 16.4 (exit). $\text{exit}(\pi, \text{code}) := \text{Collapse}\left(\text{Pop}_{\text{terminate}(\text{code})}\left(\text{Bind}_{f_{\text{valid}}}(H_\pi \oplus \{\text{continue}, \text{terminate}\})\right)\right)$

This is exit modeled exactly on the UNLINK reduction (Definition 14.2): a freshly opened option space concerning π ’s own continued standing, rather than an external relation’s. It inherits UNLINK’s non-erasure property by the same proof, and inherits UNLINK’s honesty about the collapse rule being an application-layer choice: whether a namespace renderer treats a terminated process as absent, zombie, or reapable is governed by $\mathcal{R}$ (Definition 21.8), not fixed by exit itself.

16.5 wait

Definition 16.5 (wait).

$$\text{wait}(\pi_{\text{parent}}, \pi_{\text{child}}) := \text{Collapse}\left(\text{Pop}_{\text{resume}}\left(\text{Bind}_{f_{\text{terminated}}}(H_{\pi_{\text{parent}}} \otimes H_{\pi_{\text{child}}})\right)\right)$$

where $f_{\text{terminated}}(\text{resume}) := [\exists \text{terminate}(\text{code}) \in H_{\pi_{\text{child}}}\text{-component of the joint term}]$.

Proposition 16.3 (wait Blocks Structurally, Not Operationally). $\text{resume} \notin \text{Opt}'$ after $\text{Bind}_{f_{\text{terminated}}}$ whenever π_{child} 's history contains no $\text{terminate}(\cdot)$ event, and $\text{Pop}_{\text{resume}}$ is therefore inadmissible under the pop rule until the child terminates.

Proof. By Bind 's definition (bind rule), $f_{\text{terminated}}$ filters resume out of Opt exactly when the existential condition is false, since $\text{Opt}' = \{o \in \text{Opt} : f(o)\}$. The pop rule requires its target to lie in Opt' ; $\text{resume} \notin \text{Opt}'$ makes $\text{Pop}_{\text{resume}}$ untypeable until re-evaluation of the Bind , after further child events, admits it. $\square$

Corollary 16.1 (wait Recomposes Without Inverting fork). wait 's use of $\otimes$ to join $H_{\pi_{\text{parent}}}$ and $H_{\pi_{\text{child}}}$ does not reconstruct the pre-fork H_{π} ; by Proposition 15.2, the joint term retains both processes' independent post-branch histories in full.

Proposition 16.3 is the precise sense in which wait blocks: not as an operational suspension the scheduler must special-case, but as an ordinary instance of Bind narrowing an option space to empty for the option under consideration, identical in kind to the scheduler's general admissibility filtering in Section 17 — wait is the special case of scheduling arbitration where the admissibility condition happens to be phrased over another process's history rather than over priority or resource availability.

17 Scheduling

Section 16's wait was shown, in Proposition 16.3, to be an ordinary instance of Bind narrowing an option space to exclude an inadmissible candidate — no special blocking mechanism was needed beyond what the grammar already had. Scheduling is the general case of that same pattern, applied across every runnable process rather than a single parent-child pair, and this section is correspondingly short: it generalizes rather than introduces.

17.1 The Runnable Set

Definition 17.1 (Runnable Set). At kernel state σ , let $\Pi_{\text{run}}(\sigma) \subseteq \{\pi : \pi \text{ a process}\}$ be the processes not currently blocked by an empty admissible continuation set (per Proposition 16.3 or equivalent). The scheduling option space is constructed as

$$\text{Opt}_{\text{sched}}(\sigma) = \left\{ \text{continue}_{\pi} : \pi \in \Pi_{\text{run}}(\sigma) \right\}$$

over the joint term $\bigotimes_{\pi \in \Pi_{\text{run}}(\sigma)} H_{\pi}$, the $\otimes$ -composition (Axiom 14.1, extended associatively to more than two terms) of every runnable process's history.

The joint tensor is necessary rather than cosmetic: admissibility conditions below routinely need cross-process facts (whether a resource o a candidate process wants is currently held elsewhere), which are not visible from any single H_π in isolation.

17.2 Admissibility Versus Selection

Definition 17.2 (Scheduling Reduction).

$$\text{schedule}(\sigma) := \text{Collapse} \left(\text{Refuse}_{\{\pi' : \pi' \in \text{Opt}_{\text{sched}} \setminus \text{Opt}'_{\text{sched}}\}} \left(\text{Pop}_{\text{continue}_{\pi^*}} \left(\text{Bind}_{f_{\text{elig}}} \left(\bigotimes_{\pi} H_{\pi} \right) \right) \right) \right)$$

where f_{elig} filters $\text{Opt}_{\text{sched}}$ to $\text{Opt}'_{\text{sched}}$ by dependency satisfaction, resource availability, and hard policy constraints (quota, permission), and π^* is selected from $\text{Opt}'_{\text{sched}}$ by a priority function external to the grammar.

Proposition 17.1 (Priority Is Not Admissibility). *The choice of π^* from among several candidates surviving $\text{Bind}_{f_{\text{elig}}}$ is not itself a Bind, Pop, Refuse, or Collapse operation, and requires no Refuse for the candidates in $\text{Opt}'_{\text{sched}}$ not chosen.*

Proof. By the pop rule, $\text{Pop}_a(t)$ requires only $a \in \text{Opt}'_t$; it does not require that every other admissible option be separately disposed of. This matches the worked arithmetic example precedent: at a step where both options remain admissible after filtering, only the chosen option is Popped, and the unchosen one remains an unresolved element of the residual option space rather than being Refused. A candidate $\pi' \in \text{Opt}'_{\text{sched}} \setminus \{\text{continue}_{\pi^*}\}$ therefore requires no Refuse; it remains eligible and may be selected on a future scheduling reduction. $\square$

Corollary 17.1 (Refuse Records Exactly the Filtered Set). *Definition 17.2's Refuse targets exactly $\text{Opt}_{\text{sched}} \setminus \text{Opt}'_{\text{sched}}$ — the candidates Bind's filter removed from admissibility — and no others. This is forced by the grammar, not a scheduling policy choice: Refuse's typing rule requires its target to be drawn from the original unfiltered option space precisely because that option is no longer available for Pop, matching the causal calculus's stated division of labor between Bind's filtering and Refuse's recording.*

Corollary 17.1 corrects the original specification's looser phrasing — “Refuse could record an explicitly rejected candidate when that rejection matters.” Whether a rejection “matters” is not a judgment call available to the scheduler: any process failing f_{elig} (dependency unmet, resource unavailable, quota exceeded, permission revoked) is Refused, always; any process merely outranked by π^* on priority is never Refused, always. History records every enforced exclusion and no mere non-selections, avoiding both silent rejection and an unbounded history of routine priority comparisons.

17.3 Rendering the Run Queue

Reading the current run queue or CPU allocation is a Collapse over $\bigotimes_{\pi} H_{\pi}$ exactly as every current-state read a mainstream system performs silently. What a mainstream system treats as an invisible implementation detail this specification makes an explicit, namespace-mediated operation:

a run queue view is a namespace renderer (Definition 21.5) like any other, subject to Definition 21.9 if more than one namespace observes scheduling state, and to Proposition 21.4 if that observation is to be trusted across namespaces rather than merely within one.

17.4 Scheduling and Log Sequencing Are the Same Arbitration Pattern

Proposition 17.2 (Arbiter Generalization). *The Arbiter’s role (Section 11 and following) — accepting or rejecting client proposals and assigning sequence identifiers — is an instance of Definition 17.2 with $\text{Opt}_{\text{sched}}$ replaced by the set of pending proposals, f_{elig} replaced by well-formedness and permission checking, and π^* replaced by whichever proposal is sequenced next.*

Proof. Both reduce to the same shape: Bind filters a candidate set by an admissibility condition, Pop commits one survivor, Refuse records exactly the filtered-out remainder per Corollary 17.1, and Collapse projects the result — an assigned EID in the Arbiter’s case, a scheduling decision here. Axiom 13.1 (exactly one arbiter per log) is the further constraint that exactly one entity performs this reduction for the authoritative log, which Definition 17.2 does not itself require for CPU scheduling, since multiple schedulers across isolated namespaces may run the same reduction independently over disjoint Π_{run} . $\square$

This is not merely an analogy noted in passing. It means the syscall boundary (Section 18) requires no new reduction pattern of its own — a syscall is a proposal, and that section proceeds by instantiating Definition 17.2 rather than deriving a fourth version of the same structure from scratch.

18 The Syscall Boundary

The original specification described a syscall informally: “The kernel would Bind it against permissions and current constraints, Pop the accepted transition or Refuse the proposal, and Collapse the result into the value returned to the caller.” Proposition 17.2 already showed this shape is not novel to syscalls — it is the same arbitration pattern proven for scheduling and for the Arbiter’s own sequencing role. This section instantiates that pattern precisely rather than re-deriving it, and states the one genuinely new content: what a syscall proposal is permitted to reference, which is where privilege separation actually lives in this calculus.

18.1 Syscall as a Degenerate Scheduling Reduction

Definition 18.1 (Syscall Proposal). *A syscall proposal is a tuple $p = (\pi, op, args)$: the requesting process π , a requested causal operation op (itself a Bind, Pop, Refuse, or Collapse target), and $args$ referencing only fact keys in $K_{\text{ns}(\pi)}(\sigma)$ (Definition 21.3), where $\text{ns}(\pi)$ is π ’s namespace.*

The restriction to $K_{\text{ns}(\pi)}(\sigma)$ is not a permission check; it is a well-formedness condition on what the proposal can even be built to say. A process cannot propose an operation over an object it is not authorized to observe, independent of whether that operation would otherwise be permitted — the two failure modes are kept distinct below.

Definition 18.2 (Syscall Reduction). *Let $\text{Opt}_p = \{\text{admit}_p\}$ and let $f_{\text{perm}}(\text{admit}_p)$ hold iff p is well-formed per Definition 18.1 and satisfies the kernel’s permission and capability constraints at σ . Then*

$$\text{syscall}(p) := \begin{cases} \text{Collapse}(\text{Pop}_{\text{admit}_p}(\text{Bind}_{f_{\text{perm}}}(\text{Opt}_p))) & \text{if } \text{admit}_p \in \text{Opt}'_p, \\ \text{Collapse}(\text{Refuse}_{\text{admit}_p}(\text{Bind}_{f_{\text{perm}}}(\text{Opt}_p))) & \text{otherwise.} \end{cases}$$

Proposition 18.1 (Syscall Reduction Is a Degenerate Case of Definition 17.2). *Definition 18.2 is Definition 17.2 with $|\text{Opt}_{\text{sched}}| = 1$: a single candidate, no priority selection required, admission and denial both handled by the same Bind-then-commit shape already proven for scheduling and for the Arbiter.*

Proof. Immediate by comparison of the two definitions: both filter a candidate set by an admissibility predicate via Bind, commit the admissible survivor via Pop, and record the excluded alternative via Refuse per Corollary 17.1. Where Definition 17.2 additionally selects π^* from among several survivors, Definition 18.2’s Opt_p has cardinality one, making that selection step vacuous rather than absent. $\square$

This degeneracy is why syscall denial reuses Refuse rather than introducing a distinct deny_p option: a rejected proposal is not a different kind of event from an accepted one, only a differently tagged entry in the same history, and Definition 18.2’s case split is exactly that tagging, not a fork in the underlying mechanism.

Corollary 18.1 (Concurrent Syscalls Are Ordinary Scheduling). *When multiple processes submit proposals contending for the same resource within one scheduling quantum, the reduction is not several independent applications of Definition 18.2 but a single application of the general Definition 17.2 over $\text{Opt}_{\text{sched}} = \{\text{admit}_{p_1}, \dots, \text{admit}_{p_n}\}$, with f_{elig} specialized to f_{perm} for each candidate.*

Proof. Contention for a shared resource is exactly the condition under which f_{perm} for one proposal depends on facts affected by another’s admission — the case Definition 17.1’s joint tensor $\bigotimes_{\pi} H_{\pi}$ was constructed to make visible. Evaluating each p_i against $\text{Opt}_p = \{\text{admit}_{p_i}\}$ in isolation would not see this dependency; only the joint reduction does. $\square$

18.2 Privilege Separation as Namespace Asymmetry

Proposition 18.2 (Syscalls Cross a Namespace Boundary, Not a New Boundary). *The user/kernel privilege boundary is an instance of Definition 21.5’s authorized-domain asymmetry: the kernel namespace has $K_{\text{kernel}}(\sigma) = \mathcal{K}$ (every fact key), and a user namespace i has $K_i(\sigma) \subsetneq \mathcal{K}$. Evaluating f_{perm} may reference any fact in $K_{\text{kernel}}(\sigma)$, including facts outside $K_{\text{ns}(\pi)}(\sigma)$; this does not violate namespace coherence, because Definition 21.9 constrains agreement only over $D_{ij}(\sigma) = K_i(\sigma) \cap K_j(\sigma)$, and a fact the kernel consults but never reports back into $K_i(\sigma)$ is never part of any $D_{ij}(\sigma)$ a user namespace participates in.*

Proof. f_{perm} is evaluated kernel-side, prior to and independent of Collapse’s projection into the caller’s returned value; it is not itself a namespace renderer in the sense of Definition 21.5 and is not subject to Definition 21.9. Only syscall(p)’s output, Collapse($\cdot$) as rendered back through $\pi_{\text{ns}(\pi)}$, is

a namespace-rendered fact, and it is restricted to $K_{\text{ns}(\pi)}(\sigma)$ by Definition 18.1’s own well-formedness condition on what *args* — and hence what the returned value can meaningfully describe — was permitted to reference. $\square$

This is the precise content of “privilege separation” in this calculus: not a separate mechanism bolted onto the causal core, but the ordinary consequence of $K_{\text{kernel}}(\sigma) \supsetneq K_i(\sigma)$ for every user namespace i , together with Proposition 21.4’s requirement that whatever *is* reported back must use the same version-pinned rule library the kernel used internally. A permission check consulting facts the caller can never see is not a special case requiring justification; it is the default shape of any renderer pair where one side’s authorized domain strictly contains the other’s.

19 Virtual Object Namespace

The original specification’s description of a Spherepop VFS — “a path would resolve to a history-bearing object rather than merely a mutable inode” — can now be made exact, with less new machinery than the previous sections needed. Naming is not a fourth kind of structure alongside objects, relations, and metadata; it is itself a relation, and path resolution, write, read, rename, deletion, and mount all reduce to instances of apparatus already proven in Sections 14 and 18.

19.1 Extending the State Term

The identity-preservation proofs of prior sections argued informally from “the definition does not mention I ,” which proves non-interference only given a frame property this specification has not yet stated. Fixing this requires first making I an explicit component of a causal State term, which the grammar so far has left implicit.

Definition 19.1 (State Term, Extended). *A State term is a triple $t = (I_t, \text{Opt}_t, H_t)$: immutable identity, current admissible option space, and history. Bind, Pop, and Refuse write only to Opt_t and H_t :*

$$\text{Bind}_f(t) = (I_t, \{o \in \text{Opt}_t : f(o)\}, H_t), \quad \text{Pop}_a(t) = (I_t, \text{Opt}_t \setminus \{a\}, H_t \cdot [\text{sel} : a]), \quad \text{Refuse}_a(t) = (I_t, \text{Opt}_t \setminus \{a\}, H_t \cdot [\text{rej} :$$

Sequence composes operations on one t and inherits I_t trivially. $\text{Collapse}(t) : \text{State} \rightarrow \text{Obs}$ produces no new State term, so I -preservation does not apply to its output.

Axiom 19.1 (Frame). *For $t \xrightarrow{e} t'$ where e is an instance of Bind, Pop, Refuse, Collapse, or sequence, $I_{t'} = I_t$.*

Tensor and Branch are deliberately excluded from Axiom 19.1’s scope. Tensor’s silence on identity (Proposition 14.6) and Branch’s open identity (Proposition 15.1) are not oversights Frame should paper over; they are exactly the two places a use site must supply an explicit identity policy. Frame licenses every identity proof built entirely from Bind/Pop/Refuse/Collapse/sequence without re-deriving non-interference each time; it says nothing about tensor or Branch, by design.

19.2 Paths, Naming, and Content as Fact Keys

Definition 19.2 (Path Object and Naming Relation). *A path is an object $p \in O$ of designated type Path. Naming is a relation type $\text{named} \in \mathcal{T}$: $r_{\text{named}}(p, o)$ asserts that path p currently designates object o . C_{named} , the admissibility condition for this relation type, is exclusive on its first argument: at most one $r_{\text{named}}(p, \cdot)$ may be active for a given p at once, though a given o may be the target of several distinct active $r_{\text{named}}(\cdot, o)$ (hard links).*

This is not a new mechanism. C_{named} 's exclusivity is exactly the case Definition 14.1 already anticipated — “empty when r is not exclusive with any sibling relation type” — now instantiated rather than left as an unused generality.

Definition 19.3 (Path Resolution). $\text{resolve}_\sigma(p) = o$ iff $\text{active}(\text{named}, p, o) \in D$ and $\mathcal{R}(\text{active}(\text{named}, p, o), \sigma) = \text{true}$, using the shared version-pinned rule library (Definition 21.8) and its adopted most-recent-event precedence for relation activity (Section 14). $\text{resolve}_\sigma(p)$ is undefined if no such o exists.

Definition 19.4 (Fact Key, Extended). $\mathcal{K}$ (Definition 21.6) is extended with $\text{content}(o)$ alongside $\text{exists}(o)$, $\text{rep}(o)$, $\text{active}(r, a, b)$. $\mathcal{R}(\text{content}(o), \sigma)$ is evaluated by the shared version-pinned library from H_o 's sequence of committed content events; whether successive commits replace, append, or patch the reported value is that library's choice, exactly as UNLINK's superseding rule was — not fixed by Pop or Bind themselves.

19.3 read

Definition 19.5 (read).

$$\text{read}_i(p) := \pi_i(\mathcal{R}(\text{content}(\text{resolve}_\sigma(p)), \sigma)), \quad \text{defined only if } \text{content}(\text{resolve}_\sigma(p)) \in K_i(\sigma).$$

read is a direct instance of Definition 21.5's factorization: semantic evaluation via $\mathcal{R}$, presentation via π_i , with no intermediate Obs value that a state-to-view functor would need to consume after the fact. Cross-namespace read agreement is exactly Proposition 21.4, applied to the single fact key $\text{content}(o)$.

19.4 write

Definition 19.6 (write). $\text{write}(p, \Delta)$ is a syscall proposal in the sense of Definition 18.1, with $op = \text{Pop}_{\text{content}=\Delta}$ and $f_{\text{perm}} = f_{\text{writable}}$, resolved by Definition 18.2.

Proposition 19.1 (write Non-Erasure Holds Only on Admission). $H_{\text{resolve}_\sigma(p)}$ after $\text{write}(p, \Delta)$ strictly contains $H_{\text{resolve}_\sigma(p)}$ before it if $\text{admit}_p \in \text{Opt}'_p$; otherwise $H_{\text{resolve}_\sigma(p)}$ is unchanged.

Proof. By Definition 18.2's case split: the admitted branch applies $\text{Pop}_{\text{content}=\Delta}$, appending per Definition 19.1; the refused branch applies $\text{Refuse}_{\text{admit}_p}$ to Opt_p itself, never touching $H_{\text{resolve}_\sigma(p)}$. $\square$

19.5 rename

Definition 19.7 (Atomic Composition). $\langle\langle t_1; t_2 \rangle\rangle$ denotes a compound term admitted to the log under a single EID: the Arbiter sequences it as one indivisible unit, and no Collapse computed against any intermediate point between t_1 ’s and t_2 ’s internal commits is possible. This is a new capability of the Arbiter (Axiom 13.1), not a causal or structural rule.

Definition 19.8 (rename). Let $o := \text{resolve}_\sigma(p)$, bound once, before construction.

$$\text{rename}(p, p') := \begin{cases} \langle\langle \text{UNLINK}(p, o, \text{named}); \text{UNLINK}(p', \text{resolve}_\sigma(p'), \text{named}); \text{LINK}(p', o, \text{named}) \rangle\rangle & \text{if } \text{resolve}_\sigma(p) = o \\ \langle\langle \text{UNLINK}(p, o, \text{named}); \text{LINK}(p', o, \text{named}) \rangle\rangle & \text{otherwise.} \end{cases}$$

Proposition 19.2 (rename Preserves Identity). $I(o)$ is unchanged by $\text{rename}(p, p')$.

Proof. o is bound once by Definition 19.8 and referenced, never re-resolved. Every step inside $\langle\langle \cdot \rangle\rangle$ is Bind, Pop, Refuse, or sequence (per Definitions 14.2, 14.1), so Axiom 19.1 applies directly to each, and hence to their atomic composition. $\square$

Atomicity is stated here as a capability the Arbiter must supply, not derived from anything already proven — Definition 19.7 is honestly a new primitive at the interface tier (Section 22’s “syscalls and compatibility interfaces” tier), sitting outside the causal/structural/derived-function stack rather than reducing into it.

19.6 Deletion

Corollary 19.1 (Deletion Is Path-Named UNLINK). $\text{delete}(p) := \text{UNLINK}(p, \text{resolve}_\sigma(p), \text{named})$.

By Proposition 14.5 directly, $H_{\text{resolve}_\sigma(p)}$ is untouched; only $r_{\text{named}}(p, \cdot)$ ’s activity is revoked.

No new proof is required or offered — this corollary’s entire content is that deletion needed no new apparatus, which is itself the claim worth recording. An object with every naming relation revoked becomes unreachable by path resolution while its history remains addressable by $I(o)$ directly: “revoke reachability while preserving historical existence,” now identified as literally the same operation already proven for relation revocation generally rather than a filesystem-specific behavior.

19.7 mount

Definition 19.9 (mount).

$$\text{mount}(p_{\text{point}}, o_{\text{root}}, v) := \text{Collapse}\left(\text{Refuse}_{v'}\left(\text{Pop}_{\text{mounted}(p_{\text{point}}, o_{\text{root}}, v)}\left(\text{Bind}_{f_{\text{valid}}}(H_{\text{local}} \otimes H_{o_{\text{root}}})\right)\right)\right)$$

where $\text{mounted} \in \mathcal{T}$ is exclusive on p_{point} (at most one active mount per mount point, matching named’s exclusivity discipline), f_{valid} checks p_{point} is a valid, currently-unmounted directory object, and v' ranges over any competing mount proposals for the same point.

This follows exactly the $\otimes \rightarrow \text{Bind} \rightarrow \text{Pop}$ shape (with optional Refuse) that Definition 14.1 already established, differing only in operating over a whole external namespace rather than a single relation candidate.

Definition 19.10 (Path Resolution Under Mount). *For $p = p_{\text{point}}/p_{\text{rest}}$ where $\mathcal{R}(\text{active}(\text{mounted}, p_{\text{point}}, o_{\text{root}}, v), \sigma)$ true, $\text{resolve}_\sigma(p)$ defers to the mounted namespace’s own naming relations, resolving p_{rest} relative to o_{root} .*

Proposition 19.3 (mount Preserves External Identity). *$I(o)$ is unchanged for every o reachable through the mounted subtree.*

Proof. $\otimes$ (Axiom 14.1) does not unify any object’s identity with another’s — unlike MERGE, mount never asserts $o_1 \sim o_2$ for distinct objects, so tensor’s silence on identity (Proposition 14.6) is inapplicable here rather than a gap: no identity question is being asked of tensor in the first place. The subsequent Bind, Pop, and Refuse steps preserve I directly by Axiom 19.1. $\square$

20 Modules and Drivers

Modules are the point at which design goal G1 (Deterministic Replay, Section 1) is genuinely at risk for the first time. Every prior section extended the causal basis’s reach — LINK, scheduling, syscalls, the VFS — without touching what makes Proposition 3.1 true. Modules touch it directly, because they are the mechanism by which $\mathcal{R}$ itself becomes pluggable and upgradeable, and Proposition 3.1’s proof did not account for that.

20.1 Driver as Typed Interpreter

Definition 20.1 (Driver). *A driver for device class d is a partial function $\mathcal{D}_d : \text{RawSignal} \rightarrow \text{State}$, translating a raw device signal into a term built from Pop, Refuse, Bind, Collapse, sequence, and tensor. A driver-originated event is admitted as a syscall proposal (Definition 18.1) whose requesting “process” is a kernel-designated device namespace rather than an ordinary user process, resolved by the existing Definition 18.2 without modification.*

Treating driver input as an ordinary syscall proposal is deliberate: it means device admission inherits Corollary 18.1 for free when multiple devices signal within one scheduling quantum, rather than needing a separate arbitration mechanism invented for hardware specifically.

20.2 Module Loading Does Not Enlarge the Causal Basis

Definition 20.2 (Module). *A module is a pair (id, v) contributing, to the shared rule library, some subset of: driver translators (Definition 20.1), and fact-key evaluator functions for object or relation types the module introduces.*

Proposition 20.1 (Module Loading Is Not a Fifth Primitive). *Loading (id, v) extends $\text{dom}(\mathcal{R})$ without introducing any new causal operator, judgment form, or rule schema.*

Proof. By Definition 20.2, a module contributes only evaluator functions (consumed inside Definition 21.5’s semantic-view construction) and driver translators producing terms already built from existing operators. This is the same structural-vs-causal distinction Proposition 14.1 established for $\otimes$, and the same application-layer-choice pattern Section 14 already acknowledged for collapse rules generally: modules are pluggable providers of exactly that already-licensed flexibility, extended to relation and object types beyond the kernel’s built-in set, never a change to Pop/Refuse/Bind/Collapse themselves. $\square$

20.3 Module Version Attribution and Deterministic Replay

Definition 20.3 (Modular Rule Library). $\mathcal{R} = \mathcal{R}_{\text{core}} \cup \bigcup_{(id,v) \in \text{Mod}(\sigma)} \mathcal{R}_{id,v}$, where $\text{Mod}(\sigma)$ is the set of currently loaded module-version pairs and $\mathcal{R}_{id,v}$ is (id,v) ’s evaluator contribution.

Definition 20.4 (Module-Attributed Fact Key). A fact key k is module-attributed to (id,v) at σ if $\mathcal{R}(k, \sigma)$ is computed by $\mathcal{R}_{id,v}$ rather than $\mathcal{R}_{\text{core}}$.

Axiom 20.1 (Module Attribution Recording). For every event e_k whose committed interpretation depended on a module-attributed fact key, the log records $\text{modattr}(e_k) = (id,v)$, the specific module version whose evaluator governed that interpretation at commit time.

Definition 20.5 (Module-Consistent Replay). A replay of $\ell_{\leq n}$ is module-consistent if, for every e_k with $\text{modattr}(e_k)$ recorded, the replaying kernel evaluates any fact key depending on e_k using exactly $\mathcal{R}_{\text{modattr}(e_k)}$, regardless of which module versions the replaying kernel currently has loaded.

Proposition 20.2 (Replay Determinism Requires Module-Version Pinning). Without Axiom 20.1 enforced, two replays of the same $\ell_{\leq n}$ can produce different states σ_n , contradicting Proposition 3.1.

Proof. Let module (sensor, v_1) define $\mathcal{R}_{\text{sensor}, v_1}(\text{content}(o), \sigma)$ as the raw signal value. Let event $e_k = \text{Pop}_{\text{content}=raw}$ be committed while (sensor, v_1) is $\text{Mod}(\sigma)$ ’s active version. Let (sensor, v_2) later be loaded, redefining $\mathcal{R}_{\text{sensor}, v_2}(\text{content}(o), \sigma) = \text{raw} \times \text{calibration}$. A replay performed immediately after e_k ’s commit, under $\text{Mod}(\sigma) = \{(\text{sensor}, v_1)\}$, computes $\text{content}(o) = \text{raw}$. A replay of the identical prefix $\ell_{\leq k}$ performed later, under $\text{Mod}(\sigma) = \{(\text{sensor}, v_2)\}$ and without Axiom 20.1’s pinning enforced, computes $\text{content}(o) = \text{raw} \times \text{calibration}$. Both replay the same $\ell_{\leq k}$ from the same σ_0 ; they produce different σ_k , contradicting Proposition 3.1’s claim that a log prefix uniquely determines the resulting state. $\square$

Proposition 3.1’s original proof — “all kernel transitions are pure functions of prior state and the next event... replay is deterministic” — is not wrong, but it is incomplete once modules exist: it implicitly assumed a fixed, unchanging $\mathcal{R}$, an assumption every prior section of this specification was free to make and modules are the first to break. Proposition 20.2 does not weaken that proposition; it identifies the hypothesis modules must not violate for it to keep holding, and Definition 20.5 states exactly what a replay implementation must do to satisfy it.

Corollary 20.1 (Deterministic Replay Under Modules). If every module-attributed event carries a recorded modattr per Axiom 20.1, and every replay is module-consistent per Definition 20.5, then Proposition 3.1 holds unconditionally, including across module upgrades.

Proof. Under these two conditions, every fact-key evaluation during replay uses exactly the evaluator that was in effect at commit time, regardless of what the replaying kernel currently has loaded — collapsing the two divergent branches of Proposition 20.2’s construction into one, since $\text{Mod}(\sigma)$ at replay time no longer affects the evaluator selected for any already-committed module-attributed event. $\square$

21 Permissions, Capabilities, and Namespace Isolation

Section 18’s syscall reduction referred to “the kernel’s permission and capability constraints” without defining what a capability is. This section closes that gap, then uses it to give namespace isolation the coherence guarantee it needs and honestly cannot get for free.

21.1 Capabilities as Relations

Definition 21.1 (Capability Object and Grant Relation). *A capability is an object $c \in O$ of type Capability, carrying a fixed description $\text{authorizes}(c)$: either a fact key $k \in \mathcal{K}$ or an operation-target pair $(op, target)$. $\text{grants} \in \mathcal{T}$ is a relation type; $r_{\text{grants}}(i, c)$ asserts subject i (a process or namespace identity) currently holds c . grants is not exclusive: a subject may hold several capabilities simultaneously.*

Definition 21.2 (Grant and Revoke).

$$\text{grant}(i, c) := \text{LINK}(i, c, \text{grants}), \quad \text{revoke}(i, c) := \text{UNLINK}(i, c, \text{grants}).$$

Both are direct instantiations of Definitions 14.1 and 14.2, contributing no new causal or structural machinery. grant ’s admissibility condition C_{grants} checks that the granting party itself holds authority to grant c (an ordinary Bind filter over the granting party’s own capability set); revoke inherits Proposition 14.5 directly, so a revoked capability remains in history — auditable, never erased — while ceasing to be currently active under most-recent-event precedence, exactly as a revoked LINK does.

Definition 21.3 (Authorized Fact Domain).

$$K_i(\sigma) = \{ k \in \mathcal{K} : \exists c. \mathcal{R}(\text{active}(\text{grants}, i, c), \sigma) \wedge \text{authorizes}(c) = k \}.$$

Definition 21.4 (Operation Authorization). *$\text{authorized}(i, op, target, \sigma)$ holds iff $\exists c$ with $\mathcal{R}(\text{active}(\text{grants}, i, c), \sigma)$ and $\text{authorizes}(c) = (op, target)$.*

Proposition 21.1 (f_{perm} Is Operation Authorization). *Definition 18.2’s $f_{\text{perm}}(\text{admit}_p)$, for $p = (\pi, op, args)$, is $\text{authorized}(\text{ns}(\pi), op, args, \sigma)$ conjoined with p ’s well-formedness (Definition 18.1). Similarly, write ’s f_{writable} and mount ’s f_{valid} each reduce to $\text{authorized}(\cdot)$ for their respective $(op, target)$ pairs.*

Proof. Immediate by definition: every capability-gated Bind filter introduced in Sections 18–19 was informally described as “checking permissions”; each such check is, by Definition 21.4, exactly

membership of the relevant $(op, target)$ pair in some active capability's $authorizes(c)$. No filter introduced so far required anything $authorized(\cdot)$ does not already express. $\square$

21.2 Namespace as Bundled Renderer

Definition 21.5 (Namespace Renderer). *A namespace renderer V_i factors as $V_i(\sigma) = \pi_i(\widehat{V}_i(\sigma))$, where the semantic view $\widehat{V}_i(\sigma) = \{(k, \mathcal{R}(k, \sigma)) : k \in K_i(\sigma)\}$ evaluates every authorized fact key against one shared $\mathcal{R}$, and π_i is a presentation map (naming, formatting) applied afterward. A namespace i is the pair $(K_i(\sigma), \pi_i)$, with $K_i(\sigma)$ constructed by Definition 21.3.*

Proposition 21.2 (Isolation Is Enforced at Proposal Construction, Not Only at Rendering). *A process π in namespace i cannot construct a well-formed syscall proposal referencing args outside $K_i(\sigma)$ (Definition 18.1), independent of whether $read_i$'s domain restriction (Definition 19.5) would also have blocked the corresponding read.*

Proof. Definition 18.1 restricts $args$ to $K_{ns(\pi)}(\sigma)$ as a well-formedness condition prior to and independent of f_{perm} evaluation; Definition 19.5 restricts $read_i$'s domain as a separate condition on the rendering side. Neither implies the other, and both hold simultaneously for any process operating normally within its namespace: it cannot propose what it cannot name, and cannot read what it is not authorized to have rendered, and these are two independent enforcement points rather than one mechanism described twice. $\square$

Corollary 21.1 (Revocation Narrows Isolation Boundaries in Real Time). *If $revoke(i, c)$ commits at $\sigma \rightarrow \sigma'$ and $authorizes(c) = k$, then $k \notin K_i(\sigma')$ even though $k \in K_i(\sigma)$, and any syscall proposal from a process in namespace i referencing k becomes ill-formed from σ' onward.*

Proof. Immediate from Definition 21.3 under most-recent-event precedence: $\mathcal{R}(\text{active}(\text{grants}, i, c), \sigma')$ evaluates false once $revoke(i, c)$'s UNLINK-derived supersession is the most recent event concerning (i, c) , by the same argument as Section 14's original collapse rule for revoked relations. $\square$

21.3 Namespace Isolation Requires View Coherence

Theorem 7.1 guarantees that a *single* admissible view V composes coherently with itself across time. It says nothing about two *different* views, V_i and V_j , applied to the same σ . Namespace isolation requires exactly this: different processes receiving different Collapses of the same underlying world. The guarantee this needs does not follow from Theorem 7.1 and must be established, or shown unattainable, on its own terms.

Definition 21.6 (Observable Fact Key). *An observable fact key is a term of the form $\text{exists}(o)$, $\text{rep}(o)$, or $\text{active}(r, a, b)$, for $o \in O$, $r \in \mathcal{T}$, $a, b \in O$ (extended by Definition 19.4). Let $\mathcal{K}$ denote the set of all such keys.*

Definition 21.7 (Shared Observable Domain). *For namespace renderers V_i, V_j , $D_{ij}(\sigma) = K_i(\sigma) \cap K_j(\sigma)$.*

A fact key absent from $K_i(\sigma)$ is legitimate redaction, full stop — it is not part of $D_{ij}(\sigma)$ and imposes no agreement obligation on namespace i at all, including for other facts about the same object. Only keys both namespaces are authorized to report enter the domain over which coherence is required.

Definition 21.8 (Version-Pinned Rule Library). *A rule library $\mathcal{R}$ assigns to each fact-key type a function computing its semantic value from σ . $\mathcal{R}$ is version-pinned relative to log ℓ if every event $e_k \in \ell$ carries a recorded library-version tag v_k , and evaluating any fact key at prefix $\ell_{\leq n}$ uses exactly the rule definitions in effect at v_n — never a later library’s reinterpretation of earlier events.*

Definition 21.9 (Compatibility Relation). $V_i(\sigma) \upharpoonright_{D_{ij}(\sigma)} \sim V_j(\sigma) \upharpoonright_{D_{ij}(\sigma)}$ iff $\widehat{V}_i(\sigma) \upharpoonright_{D_{ij}(\sigma)} = \widehat{V}_j(\sigma) \upharpoonright_{D_{ij}(\sigma)}$ (exact semantic equality) and π_i, π_j differ, on that equal semantic content, only in naming or format.

Proposition 21.3 (Determinism and Shared Source Do Not Imply Coherence). *There exist admissible namespace renderers V_i, V_j and a state σ with $\text{active}(r, a, b) \in D_{ij}(\sigma)$ on which they disagree.*

Proof. $H_{a,b}$ contains $\text{Pop}_{r(a,b)}$ then $\text{Pop}_{\text{revoke}_r}$. $\mathcal{R}_i$ evaluates $\text{active}(r, a, b)$ by most-recent-event precedence (false); $\mathcal{R}_j$ by first-committed-event precedence (true). Both are functors on $\mathbf{Pref}(\ell)$, both non-interfering, hence both admissible under Definition 7.3 — that definition fixes only functoriality and non-interference, neither a canonical current-state interpretation nor revocation-sensitivity. $\text{active}(r, a, b) \in K_i(\sigma) \cap K_j(\sigma) = D_{ij}(\sigma)$ by hypothesis that both namespaces are authorized to report relation activity. The two libraries disagree on its value. $\square$

This is not a pathological edge case. It is the generic situation whenever two namespace renderers are permitted to instantiate their own collapse rules independently, which nothing prior forbids. Determinism per renderer and a shared source were the only hypotheses Theorem 7.1 required; Proposition 21.3 shows they are not enough once more than one renderer is in play.

Definition 21.10 (Namespace-Coherent Renderer Family). *A family $\{V_i\}$ is namespace-coherent if, for every i, j and every σ , $V_i(\sigma) \upharpoonright_{D_{ij}(\sigma)} \sim V_j(\sigma) \upharpoonright_{D_{ij}(\sigma)}$.*

Theorem 21.1 (Conditional Namespace Isolation Safety). *If $\{V_i\}$ is namespace-coherent, then no two namespaces derive incompatible causal claims about any shared visible object.*

Proof. Immediate from Definition 21.10: $\sim$ is defined to permit exactly the legitimate divergences of Definition 21.9 and no others, so namespace-coherence is the absence of incompatible claims on $D_{ij}(\sigma)$, restated. The content of this theorem is not in its proof but in what it refuses to claim: it does not assert that isolation, as architecturally proposed, automatically produces a namespace-coherent family. Proposition 21.3 already shows it does not, in general. $\square$

Proposition 21.4 (Shared-Library Divergence Suffices). *If every namespace renderer uses the same version-pinned $\mathcal{R}$ (Definition 21.8) to compute every shared observable fact, and namespaces differ only in $K_i(\sigma)$ and π_i , then $\{V_i\}$ is namespace-coherent, with $\widehat{V}_i(\sigma) \upharpoonright_{D_{ij}(\sigma)} = \widehat{V}_j(\sigma) \upharpoonright_{D_{ij}(\sigma)}$ exactly, prior to any presentation-level comparison via $\sim$.*

Proof. For $k \in D_{ij}(\sigma)$, both $\hat{V}_i(\sigma)$ and $\hat{V}_j(\sigma)$ evaluate k as $\mathcal{R}(k, \sigma)$, the same function applied to the same σ under the same pinned version, by hypothesis. Equality follows key-by-key, hence as sets over $D_{ij}(\sigma)$. π_i, π_j act only afterward, on already-equal semantic content, so any resulting difference is representational, satisfying Definition 21.9 via $\sim$ trivially once semantic equality is established. $\square$

Proposition 21.4 is the operational payoff: it turns “coherence must be imposed as an admissibility condition” into a concrete, enforceable design rule — namespace renderers must share one canonical, version-pinned collapse-rule library (Section 20) and diverge only by authorized-domain masking (Section 21.1–21.2), never by redefining what a relation or object evaluates to.

22 Architectural Layering

22.1 The Five-Way Taxonomy

Spherepop OS distinguishes causal operators, structural rules, derived kernel functions, interface operations, and non-causal annotations. Pop, Refuse, Bind, and Collapse constitute the causal basis. Sequence, tensor, and Branch construct the terms over which that basis operates. Process management, scheduling, namespace operations, messaging, and revocation are derived kernel functions. Syscalls expose those functions across the privilege boundary. Metadata remains a separate annotation channel and does not participate in causal reduction.

22.2 Where Everything Landed

Tier	Members
Causal operators	Pop, Refuse, Bind, Collapse
Structural rules	sequence, tensor ($\otimes$), Branch
Derived kernel functions	LINK, UNLINK, fork, exec, exit, wait, schedule, write, read, rename, delete, mount, grant, revoke
Hybrid (structural + external identity policy)	MERGE, bulk COLLAPSE(S, o_r)
Interface operations	syscall boundary, Atomic Composition ($\langle\langle\cdot\rangle\rangle$), driver admission
Non-causal annotations	SET_META, layout hints, debugging labels, presentation maps (π_i)

Two placements are worth flagging rather than leaving implicit in the table. Atomic Composition sits in the interface tier, not the derived tier, because Section 19 showed it is not reducible to the causal or structural rules — it is a new capability the Arbiter must supply, honestly placed outside the reduction rather than forced into it. Presentation maps sit in the annotation tier alongside metadata, not the derived tier, because Definition 21.5 factored every namespace renderer into a

semantic evaluation (causal, via $\mathcal{R}$) and a presentation step (non-causal, via π_i) precisely to keep these two kinds of difference from being confused with each other, per Definition 21.9.

22.3 Comparative Rationale

The comparison to POSIX kernels, CRDT-based systems, and version control can now be stated against the taxonomy rather than in general terms.

Unlike POSIX kernels, every derived kernel function in this specification is a typed composition traceable to four causal operators and three structural rules, rather than an opaque syscall whose internal state mutation is invisible outside a debugger; Section 14’s reduction table and Sections 16–21’s extensions are the traceability, not a claim about it.

Unlike CRDT-based systems, a single total order is enforced at every tier — Axiom 13.1 governs not only ordinary events but the Atomic Composition construct of Section 19, which exists specifically because even an interface-layer convenience must still resolve to one position in one total order, not a conflict-free merge of concurrent writes.

Unlike version control systems, speculation is structurally identical to ordinary branching rather than a separate workflow: Section 15 showed the Speculative Branch mechanism (Section 11) is Definition 15.1 under a different name, meaning a Spheredpop OS process’s fork and a client’s speculative overlay are the same term at the same tier, differing only in which identity policy (Section 16’s fork policy vs. speculation’s identity-preserving policy) governs their two projections.

22.4 What This Specification Establishes, Assumes, and Leaves Open

Established unconditionally. Theorem 14.1 (the four-primitive causal basis is complete for causal history transformation, precisely scoped to exclude SET_META); Proposition 14.1 and Proposition 15.2 (tensor and Branch are structural, not causal, and are not inverse); non-erasure for UNLINK, exec, and write on its admitted branch (Propositions 14.5, 16.2, 19.1); every identity-preservation result built from Axiom 19.1 (rename, mount, LINK, UNLINK).

Established conditionally. Theorem 21.1 holds given a namespace-coherent renderer family — Proposition 21.4 gives one sufficient, checkable way to satisfy that hypothesis, but does not claim it is the only way, and Proposition 21.3 shows the hypothesis cannot simply be assumed. Corollary 20.1 holds given Axiom 20.1 is actually enforced by every module and every replay implementation — a real deployment obligation, not a consequence of the grammar alone. rename’s atomicity holds given Definition 19.7’s Arbiter capability is implemented, which Section 19 was explicit is a new primitive supplied at the interface tier, not derived from anything proven earlier.

Left open. The priority function selecting π^* among several admissible scheduling candidates (Proposition 17.1 deliberately leaves this external); the concrete replace/append/patch semantics for any given content fact key (deferred to $\mathcal{R}$ ’s design per object type, exactly as UNLINK’s supersession rule was deferred in Section 14); reclamation of history behind permanently Refused or superseded branches — a cost the retention discipline incurs without a matching reclamation story, still unaddressed by anything in this specification; the identity-designation policy MERGE and bulk COLLAPSE both require beyond the causal-plus-structural reduction (Propositions 14.6 and 14.3) — this specification fixes one instance of that policy operationally (union–find representative assignment)

but does not state the policy in general, leaving open whether a merged or collapsed identity must always be selected from among its inputs or may in principle be freshly minted; and a conflict-resolution procedure for competing LINK or mount proposals beyond “the losing candidates are Refused,” which says what is recorded but not how contention among several simultaneously valid candidates is adjudicated when priority, not mere admissibility, is what distinguishes them.

22.5 Conclusion

Spherepop OS presents a disciplined rethinking of operating system structure grounded in determinism, causal clarity, and semantic primacy. Sections 14–21 add to the original specification not a change of ambition but a change of evidential standard: every Linux-like capability absorbed into this design — processes, scheduling, a syscall boundary, a virtual namespace, modules, permissions — is now either a proven composition over four causal primitives and three structural rules, or an explicitly named exception to that reduction, rather than an architectural aspiration stated in prose and left there. The system’s future extensions, including richer semantic types, entropy-driven scheduling, and distributed arbitration, inherit a taxonomy precise enough to say, of any proposed addition, exactly which tier it belongs to and what obligations that placement carries.